

Securite des Applications Mobiles

MLIAEdu — Universite Cadi Ayyad

LAB 02

Rooting Android

Auteur : Chagdaly Hiba

Date : 07 mai 2026

Environnement : Mobexler + Genymotion (Android 10 — API 29)

Table des matières

1	Perimetre du laboratoire	2
1.1	Objectif	2
1.2	Regles appliquees	2
2	Definition du rooting	2
3	Verified Boot et AVB	2
3.1	Verified Boot	2
3.2	Android Verified Boot (AVB)	3
3.3	Observation sur Genymotion	3
4	Etapes du laboratoire	3
4.1	Etape 1 — Connexion et verification de l'environnement	3
4.2	Etape 2 — Activation du mode root	4
4.3	Etape 3 — Verification des privileges root	4
4.4	Etape 4 — Desactivation de Verity	5
4.5	Etape 5 — Verification de l'etat initial du device	5
4.6	Etape 6 — Installation et lancement de l'application de test	6
4.7	Etape 7 — Inspection des donnees de l'application (MASTG)	7
4.8	Etape 8 — Analyse des journaux systeme (MASTG)	8
5	Scenarios de test	9
6	Matrice de risques	9
7	Mesures defensives	10
8	Referentiels OWASP	10
8.1	MASVS — Exigences applicables	10
8.2	MASTG — Tests appliques	10
9	Remise a zero de l'environnement	11
10	Checklist finale	11
10.1	Debut de seance	11
10.2	Fin de seance	11
11	Avertissement legal	11

Perimetre du laboratoire

Parametre	Valeur
Date	07 mai 2026
Auteur	Chagdaly Hiba
Support	Genymotion Phone_2 (emulateur)
Version Android	10.0 — API 29
Architecture	x86
Application	OWASP MSTG UnCrackable Level 1
Donnees	Fictives uniquement
Reseau	Environnement de test isole
Reset effectue	Oui

TABLE 1 – Fiche perimetre du laboratoire

Objectif

Ce laboratoire a pour objectif de comprendre le concept de rooting sur Android, d’observer ses impacts sur les mecanismes de securite du systeme, et de documenter l’ensemble de la demarche avec tracabilite complete.

Regles appliquees

- Tests effectues uniquement sur un emulateur dedie
- Donnees fictives exclusivement
- Aucun compte personnel utilise
- Reset obligatoire en fin de seance
- Environnement reseau isole

Definition du rooting

Le rooting designe l’obtention des privileges superutilisateur (root) sur un systeme Android. Cette operation modifie le modele de confiance et les protections de securite du systeme d’exploitation. En contexte de laboratoire, elle permet d’observer des comportements systeme normalement inaccessibles aux utilisateurs standards. Toute operation de rooting necessite un isolement strict, une tracabilite complete et une remise a zero de l’environnement.

Verified Boot et AVB

Verified Boot

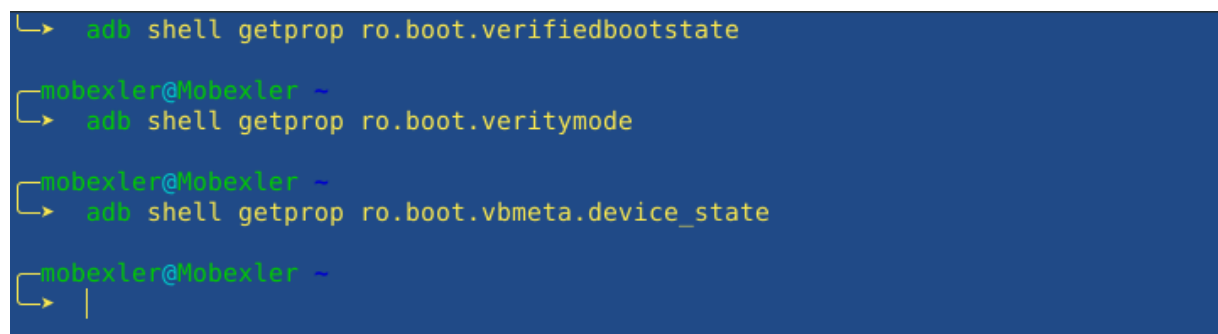
Verified Boot garantit que le systeme charge au demarrage est celui prevu par le fabricant, sans modification malveillante. Il repose sur une chaine de confiance ou chaque composant verifie l’authenticite du suivant avant de lui accorder l’execution.

Android Verified Boot (AVB)

AVB est l'implementation moderne de Verified Boot. Il ajoute une verification d'integrite pour chaque partition et inclut une protection anti-rollback pour empecher l'installation de versions systeme anterieures et vulnerables.

Observation sur Genymotion

Les proprietes `ro.boot.verifiedbootstate`, `ro.boot.veritymode` et `ro.boot.vbmeta.device_state` ont retourne des valeurs vides sur Genymotion. Ce comportement est attendu : Genymotion n'implemente pas AVB en tant qu'emulateur logiciel. Sur un appareil reel, `ro.boot.verifiedbootstate` retournerait `green` (integre), `orange` (modifie) ou `red` (compromis).



```
└─> adb shell getprop ro.boot.verifiedbootstate
mobexler@Mobexler ~
└─> adb shell getprop ro.boot.veritymode
mobexler@Mobexler ~
└─> adb shell getprop ro.boot.vbmeta.device_state
mobexler@Mobexler ~
└─> |
```

FIGURE 1 – Proprietes Verified Boot — non implementees sur Genymotion

Etapas du laboratoire

Etape 1 — Connexion et verification de l'environnement

```
adb connect 192.168.86.101:5555
adb devices
```

Listing 1 – Connexion ADB a Genymotion

```
mobexler@Mobexler ~  
└─> adb connect 192.168.86.101:5555  
* daemon not running; starting now at tcp:5037  
* daemon started successfully  
connected to 192.168.86.101:5555  
mobexler@Mobexler ~  
└─> adb devices  
List of devices attached  
192.168.86.101:5555    device
```

FIGURE 2 – Genymotion detecte via ADB — etat : device

Etape 2 — Activation du mode root

```
adb root  
adb remount
```

Listing 2 – Activation root et remontage systeme

```
mobexler@Mobexler ~  
└─> adb root  
adbd is already running as root  
mobexler@Mobexler ~  
└─> adb remount  
remount succeeded
```

FIGURE 3 – ADB en mode root — remount reussi

Etape 3 — Verification des privileges root

```
adb shell id
```

Listing 3 – Verification des privileges root

```
mobexler@Mobexler ~  
└─> adb shell id  
uid=0(root) gid=0(root) groups=0(root),1004(input),1007(log),1011(adb),1015(sdcard_rw),1028(sdcard_r)  
,3001(net_bt_admin),3002(net_bt),3003(inet),3006(net_bw_stats),3009(readproc),3011(uhid) context=u:r:  
su:s0  
mobexler@Mobexler ~  
└─> adb shell getprop ro.product.cpu.abi  
x86
```

FIGURE 4 – Confirmation root : uid=0(root)

Etape 4 — Desactivation de Verity

```
adb disable-verity
```

Listing 4 – Desactivation de dm-verity

```
mobexler@Mobexler ~  
└─> adb disable-verity
```

FIGURE 5 – Desactivation de verity executee

Etape 5 — Verification de l'etat initial du device

Avant tout test, l'ecran d'accueil du device est verifie : aucun compte personnel, aucune application residuelle.

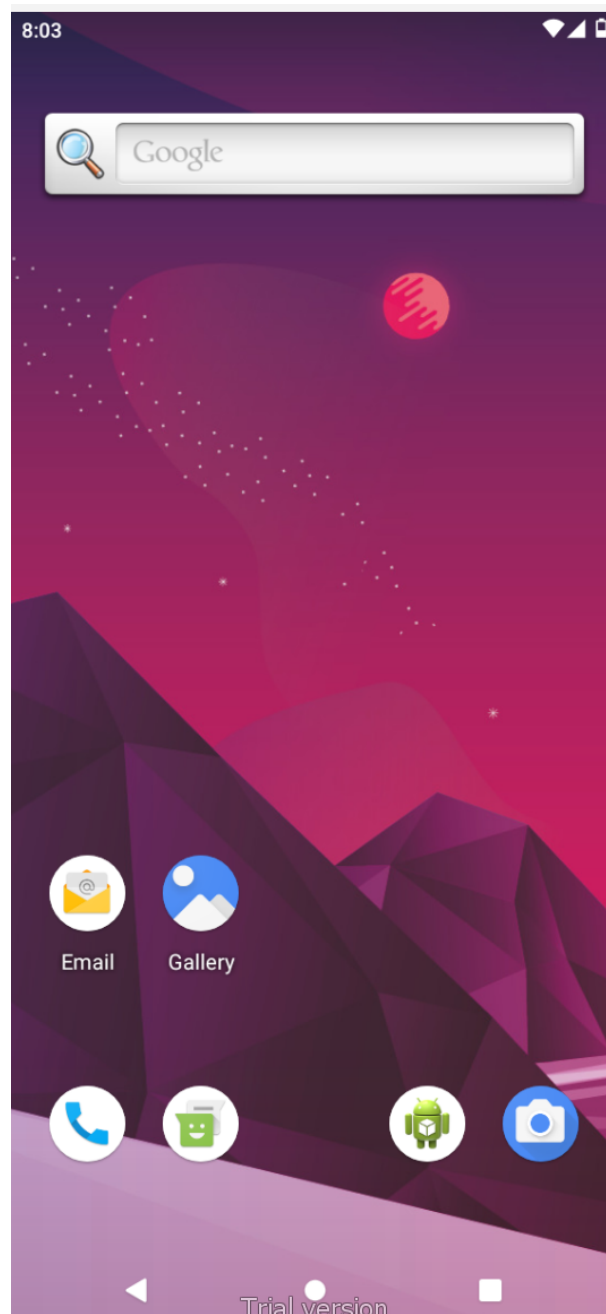


FIGURE 6 – Ecran d'accueil Genymotion — device propre

Etape 6 — Installation et lancement de l'application de test

L'application OWASP MSTG UnCrackable Level 1 etait deja presente sur le device Genymotion. Elle a ete lancee via la commande suivante :

```
adb shell monkey -p owasp.mstg.uncrackable1 1
```

Listing 5 – Lancement de l'application de test

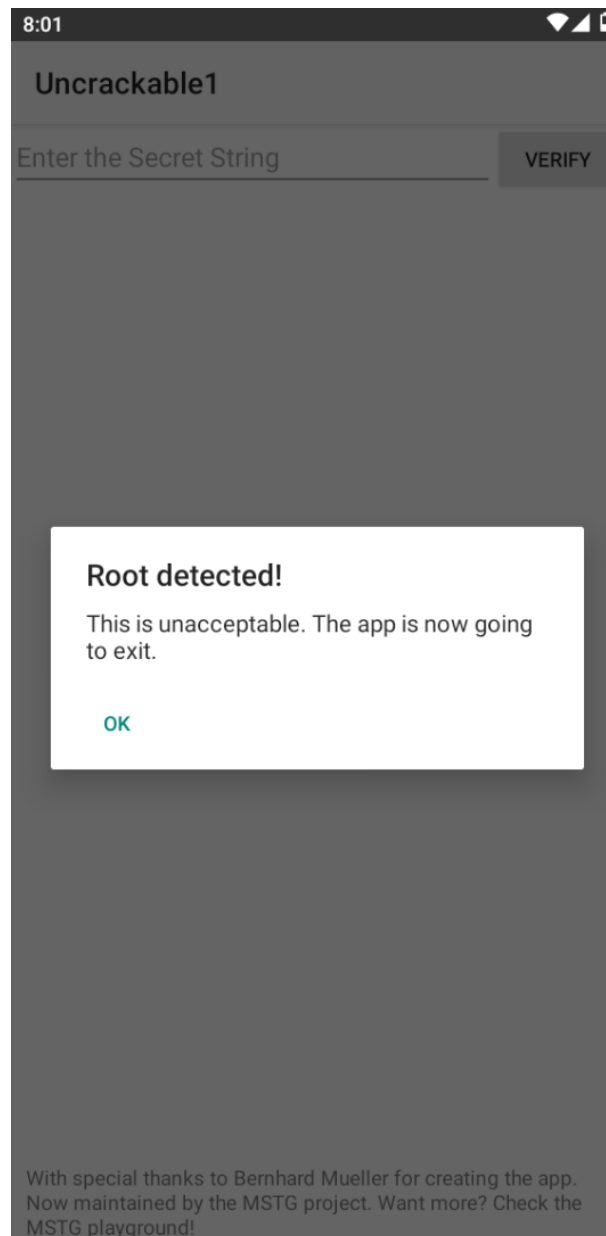


FIGURE 7 – UnCrackable1 — Detection du root confirmée

L'application a immédiatement affiché une alerte *"Root detected! This is unacceptable. The app is now going to exit."* Cette réaction confirme que le device est correctement configuré en mode root et que l'application intègre un mécanisme de détection actif.

Etape 7 — Inspection des données de l'application (MASTG)

```
adb shell ls /data/data/owasp.mstg.uncrackable1/
```

Listing 6 – Inspection des données privées de l'app


```
mobexler@Mobexler ~  
└─> adb shell ls /data/data/owasp.mstg.uncrackable1/  
cache  
code_cache  
files
```

FIGURE 8 – Contenu du repertoire de donnees de l'application

Observation : aucun repertoire `shared_prefs` n'a ete trouve. L'application ne stocke aucune donnee en clair dans les preferences partagees. Seuls les repertoires `cache`, `code_cache` et `files` sont presents.

Etape 8 — Analyse des journaux systeme (MASTG)

```
adb logcat -d | grep -i uncrackable | tail -n 20  
adb logcat -d | tail -n 200 > logcat_root_check.txt  
cat logcat_root_check.txt
```

Listing 7 – Analyse logcat

```
mobexler@Mobexler ~  
└─> adb logcat -d | grep -i uncrackable | tail -n 20  
05-07 07:52:59.189 2737 2737 W Monkey : args: [-p, package:owasp.mstg.uncrackable1, 1]  
05-07 07:52:59.194 2737 2737 W Monkey : arg: "package:owasp.mstg.uncrackable1"  
05-07 07:52:59.194 2737 2737 W Monkey : data="package:owasp.mstg.uncrackable1"  
05-07 08:00:30.790 2775 2775 W Monkey : args: [-p, owasp.mstg.uncrackable1, 1]  
05-07 08:00:30.791 2775 2775 W Monkey : arg: "owasp.mstg.uncrackable1"  
05-07 08:00:30.792 2775 2775 W Monkey : data="owasp.mstg.uncrackable1"  
05-07 08:00:30.835 756 1778 I ActivityTaskManager: START u0 {act=android.intent.action.MAIN cat=[an  
roid.intent.category.LAUNCHER] flg=0x10200000 cmp=owasp.mstg.uncrackable1/sg.vantagepoint.uncrackable1  
MainActivity} from uid 0  
05-07 08:00:30.986 2790 2790 W tg.uncrackable: Unexpected CPU variant for X86 using defaults: x86  
05-07 08:00:31.010 756 787 I ActivityManager: Start proc 2790:owasp.mstg.uncrackable1/u0a106 for a  
ctivity {owasp.mstg.uncrackable1/sg.vantagepoint.uncrackable1.MainActivity}
```

FIGURE 9 – Journaux systeme — lancement depuis uid 0 (root)

```

mobexler@Mobexler ~
└─ cat logcat_root_check.txt
05-07 08:16:20.711 2263 2383 E SQLiteDatabase:      at android.content.ContentProvider.applyBatch(C
ontentProvider.java:2126)
05-07 08:16:20.711 2263 2383 E SQLiteDatabase:      at com.android.providers.media.MediaProvider.ap
plyBatch(MediaProvider.java:3302)
05-07 08:16:20.711 2263 2383 E SQLiteDatabase:      at android.content.ContentProvider.applyBatch(C
ontentProvider.java:2117)
05-07 08:16:20.711 2263 2383 E SQLiteDatabase:      at android.content.ContentResolver.applyBatch(C
ontentResolver.java:1865)
05-07 08:16:20.711 2263 2383 E SQLiteDatabase:      at com.android.providers.media.scan.ModernMedia
Scanner$Scan.applyPending(ModernMediaScanner.java:439)
05-07 08:16:20.711 2263 2383 E SQLiteDatabase:      at com.android.providers.media.scan.ModernMedia
Scanner$Scan.maybeApplyPending(ModernMediaScanner.java:432)
05-07 08:16:20.711 2263 2383 E SQLiteDatabase:      at com.android.providers.media.scan.ModernMedia
Scanner$Scan.visitFile(ModernMediaScanner.java:412)
05-07 08:16:20.711 2263 2383 E SQLiteDatabase:      at com.android.providers.media.scan.ModernMedia
Scanner$Scan.visitFile(ModernMediaScanner.java:207)
05-07 08:16:20.711 2263 2383 E SQLiteDatabase:      at java.nio.file.Files.walkFileTree(Files.java:
2670)
05-07 08:16:20.711 2263 2383 E SQLiteDatabase:      at java.nio.file.Files.walkFileTree(Files.java:
2742)

```

FIGURE 10 – Contenu du fichier logcat_root_check.txt

Observation : les journaux confirment le lancement de l'application depuis uid 0 (contexte root) et la detection du root par l'application.

Scenarios de test

#	Scenario	Resultat observe
1	Lancer l'application	Application demarree via ADB monkey
2	Observer la detection root	Alerte "Root detected" affichee imme- diatement
3	Inspecter les donnees	Aucun shared_prefs — pas de donnees en clair

TABLE 2 – Scenarios de test executes

Matrice de risques

#	Risque
1	Integrite non garantie — conclusions potentiellement biaisees
2	Surface d'attaque accrue si l'appareil sort du laboratoire
3	Donnees sensibles exposees si presentes sur le device
4	Instabilite systeme — tests potentiellement non reproductibles
5	Melange comptes personnels et de test — fuite possible d'informa- tions
6	Mauvais nettoyage en fin de seance — persistance de donnees sen- sibles
7	Reseau non isole — effets involontaires sur des systemes externes
8	Tracabilite insuffisante — impossible de reproduire ou d'auditer

#	Risque
---	--------

TABLE 3: Matrice des risques identifiées

Mesures defensives

#	Mesure
1	Reseau isole — aucune communication non controlee
2	Donnees fictives uniquement — risque de fuite elimine
3	Device / emulateur dedie exclusivement aux tests de securite
4	Snapshot ou wipe en fin de seance
5	Journal de configuration detaille pour assurer la reproductibilite
6	Aucun compte personnel utilise
7	Controle strict des APK installees
8	Horodatage et captures a chaque etape

TABLE 4: Mesures defensives appliquees

Referentiels OWASP

MASVS — Exigences applicables

Exigence	Description
STORAGE-1	Les donnees sensibles (cles API, mots de passe, tokens) doivent etre stockees de maniere chiffree en utilisant des fonctions de chiffrement appropriees.
NETWORK-1	Les communications reseau doivent utiliser TLS avec une configuration correcte et une verification des certificats.

TABLE 5 – Exigences OWASP MASVS pertinentes

MASTG — Tests appliques

Test	Commande / Action
Verifier les preferences partagees	<code>adb shell ls /data/data/[pkg]/shared_prefs/</code>
Analyser les journaux pour fuites	<code>adb logcat -d grep -i uncrackable</code>

TABLE 6 – Tests OWASP MASTG appliques

Remise a zero de l'environnement

La remise a zero a ete effectuee via l'option *Factory reset* du gestionnaire de devices Genymotion. Cette operation efface l'ensemble des donnees utilisateur et restaure le device dans son etat initial.



FIGURE 11 – Menu Factory Reset Genymotion

Checklist finale

Debut de seance

- Perimetre documente
- Device Genymotion propre (Android 10 — API 29)
- Application de test disponible
- 3 scenarios documentes
- Versions Android et application notees

Fin de seance

- Donnees de test supprimees
- Factory reset effectue
- Preuve du reset capturee
- Rapport et tracabilite sauvegardes
- Aucun compte personnel utilise

Avertissement legal

L'ensemble des tests de ce laboratoire a ete realise dans un environnement isole et autorise, a des fins pedagogiques uniquement. Le rooting d'un appareil personnel peut invalider la garantie constructeur, exposer des donnees personnelles, et dans certaines juridictions constituer une violation des conditions d'utilisation ou des reglementations en vigueur.